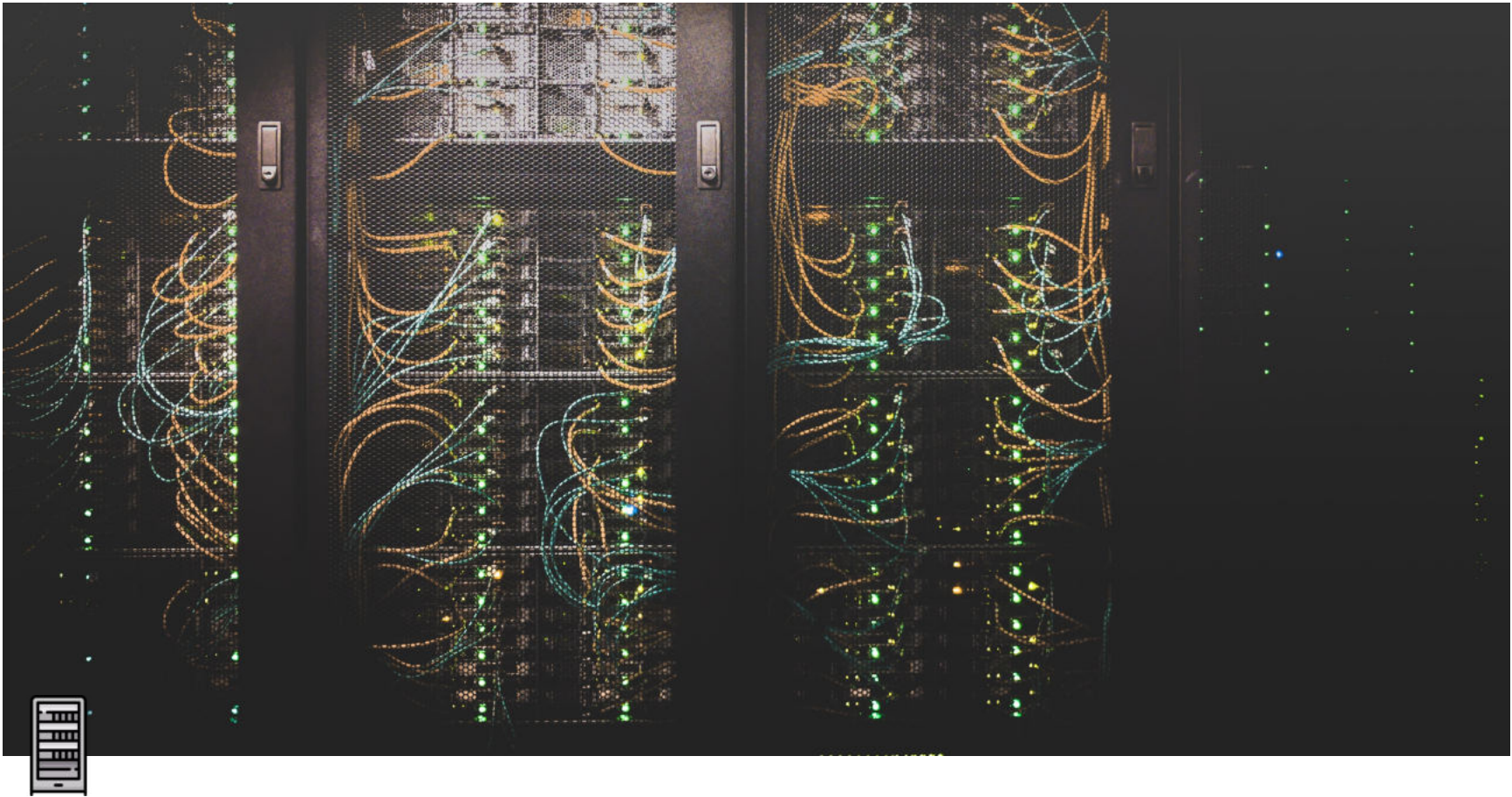
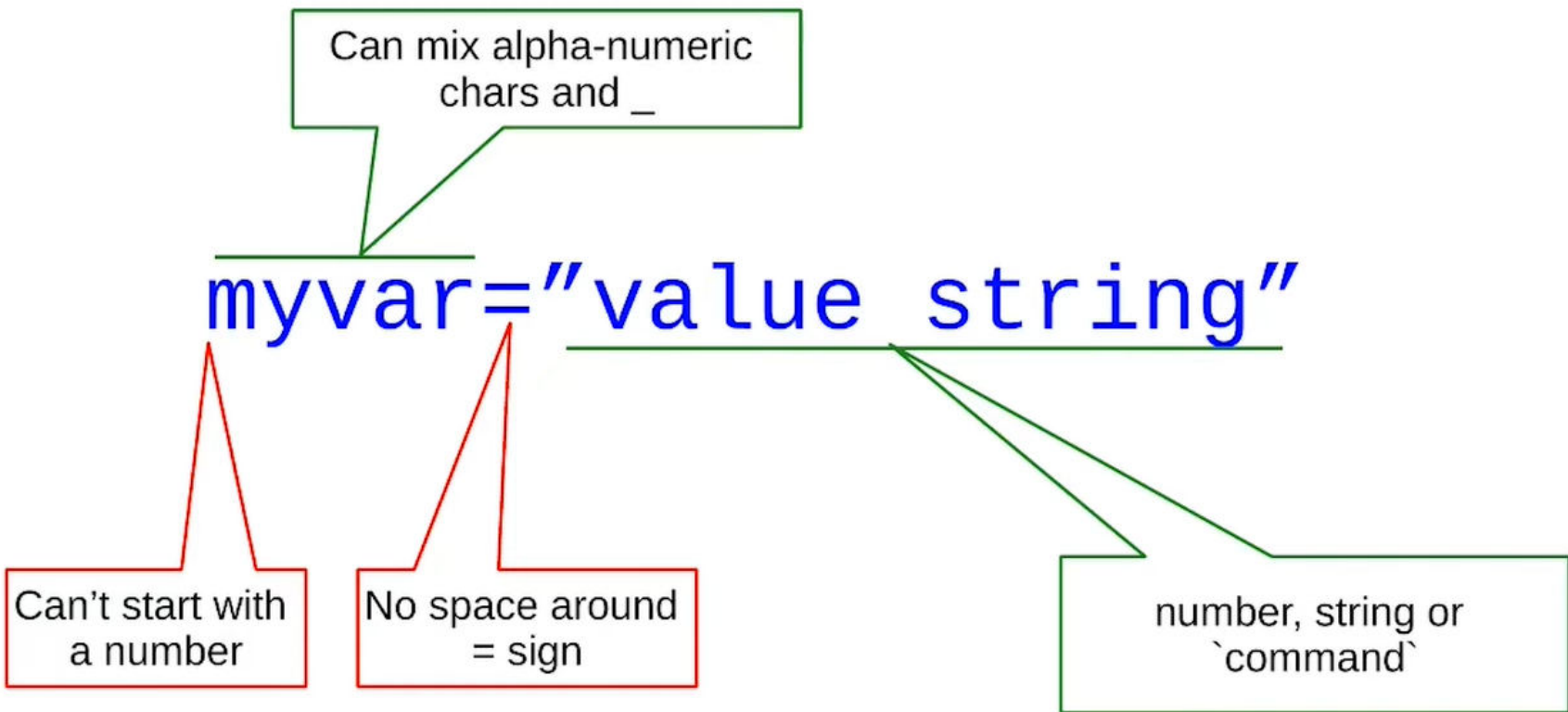




Shell variables

Type	Lecture
Date	@January 9, 2022
Lecture #	3
Lecture URL	https://youtu.be/QX5XEIFRpck
Notion URL	https://21f1003586.notion.site/Shell-variables-7fc4b392e99b4d59a13eebefdae88e1e
Week #	3

Creating a variable



Exporting a variable

Exporting means making the value of the variable available to a shell spawned by the current shell (wut)

```
export myvar="value string"
```

OR

```
myvar="value string"
export myvar
```

Using variable values

```
echo $myvar
```

```
echo ${myvar}
```

```
echo "${myvar}"
```

Removing a variable

```
unset myvar
```

Removing value of a variable

```
myvar=
```

Test if a variable is set

```
[[ -v myvar ]];
```

```
echo $?
```

Return codes:

0 → success (variable `myvar` is set)

1 → failure (variable `myvar` is not set)

Test if a variable is *not* set

```
[[ -z ${myvar+x} ]];
```

Here, *x* can be any string

```
echo $?
```

Return codes:

0 → success (variable `myvar` is not set)

1 → failure (variable `myvar` is set)

Substitute default value

If the variable `myvar` is not set, use `"default"` as its default value

```
echo ${myvar:-"default"}
```

So, if `myvar` is set, display its value else display `"default"`

Reset value if variable is set

If the variable `myvar` is set, then set `"default"` as its value

```
echo ${myvar:+"default"}
```

So, if `myvar` is set, change it's value to `"default"` and display it else display null

List of variable names

```
echo ${!H*}
```

List of names of shell variables that start with `H`

Length of string value

```
echo ${#myvar}
```

Display length of the string value of the variable `myvar`

If `myvar` is not set, display 0

Slice of a string value

```
echo ${myvar:5:4}
```

Display 4 chars of the string value of the variable `myvar`, skipping first 5 chars

Remove matching pattern

```
echo ${myvar#pattern} → match once
```

```
echo ${myvar##pattern} → match max possible
```

Keep matching pattern

```
echo ${myvar%pattern} → match once
```

`echo ${myvar%%pattern}` → *match max possible*

Replace matching pattern

`echo ${myvar/pattern/string}` → *match once and replace with `string`*

`echo ${myvar//pattern/string}` → *match max possible and replace with `string`*

Replace matching pattern by location

`echo ${myvar/#pattern/string}` → *match at beginning and replace with `string`*

`echo ${myvar/%pattern/string}` → *match at the end and replace with `string`*

Changing case

`echo ${myvar,}` → *change the first char to lower case*

`echo ${myvar,,}` → *change all chars to lower case*

`echo ${myvar^}` → *change first char to upper case*

`echo ${myvar^^}` → *change all chars to upper case*

Restricting value types

`declare -i myvar` → *only integers can be assigned*

`declare -l myvar` → *only lower case chars can be assigned*

`declare -u myvar` → *only upper case chars can be assigned*

`declare -r myvar` → *variable is read-only*

You can remove the restrictions by replacing the `-` sign with a `+` sign

However, `declare +r myvar` will **NOT** work

Indexed arrays

`declare -a arr` → *declare `arr` as an indexed array*

`$arr[0]="value"` → *set value of element with index 0 in the array*

`echo ${arr[0]}` → *value of the element at index 0 of array*

`echo ${#arr[@]}` → *number of elements in the array*

`echo ${!arr[@]}` → *display all the indices used*

`echo ${arr[@]}` → *display values of all elements of the array*

`unset 'arr[2]'` → *delete element with index 2 in the array*

`arr+=("value")` → *append an element with a value to the end of the array*

Associative arrays

Kind of like Hash maps?

`declare -A hash` → *declare `hash` as an associative array*

`$hash["a"]="value"` → *set value of element with index "a" in the array*

`echo ${hash["a"]}` → *value of element with index (or key?) "a" in the array*

`echo ${#hash[@]}` → *number of elements in the array*

`echo ${!hash[@]}` → *display all indices used*

`echo ${hash[@]}` → *display values of all elements of the array*

`unset 'hash["a"]'` → *delete element with index "a" in the array*